

Spine Formal-v6 Primary Dynamic-Graph Results

Correctness-Gated Spine vs. GraSU+ReGraph K4-shared

Chuxiao Han

Live snapshot, July 2026

Abstract

This live report contains 12 of 34 expected formal-v6 executions and 1 complete Spine/K4-shared pairs. Missing bars are still-running or queued executions, never zero-valued measurements. Every admitted row passes architecture-precision and independent mathematical oracles, full final-state comparison, and request, response, and DRAM conservation.

1 Measurement contract

Algorithm	System	Window	Warm state
CC	Spine	complete_execution	False
ResPR	Spine	complete_execution	False
SSSP	G+R K4-shared	complete_execution	False
SSSP	Spine	dynamic_update_only	True

Table 1: Measurement windows observed in admitted formal-v6 rows.

SSSP interpretation. Spine starts from a verified persisted old-graph SSSP state and measures the accepted update, automatic active-source discovery, incremental propagation, and drain. The conversion-free GraSU+ReGraph baseline performs its declared complete ReGraph execution because it has no equivalent persisted incremental SSSP state. This is an architecture-level dynamic-service comparison, not an identical-kernel microbenchmark.

Claim boundary. The report is partial until all expected rows finish. Device cycles, accepted memory bytes, and DRAMSim3 HBM energy are simulator outputs. They do not claim cycle-for-cycle FPGA calibration, on-chip dynamic energy, or total board power.

2 Current primary comparison

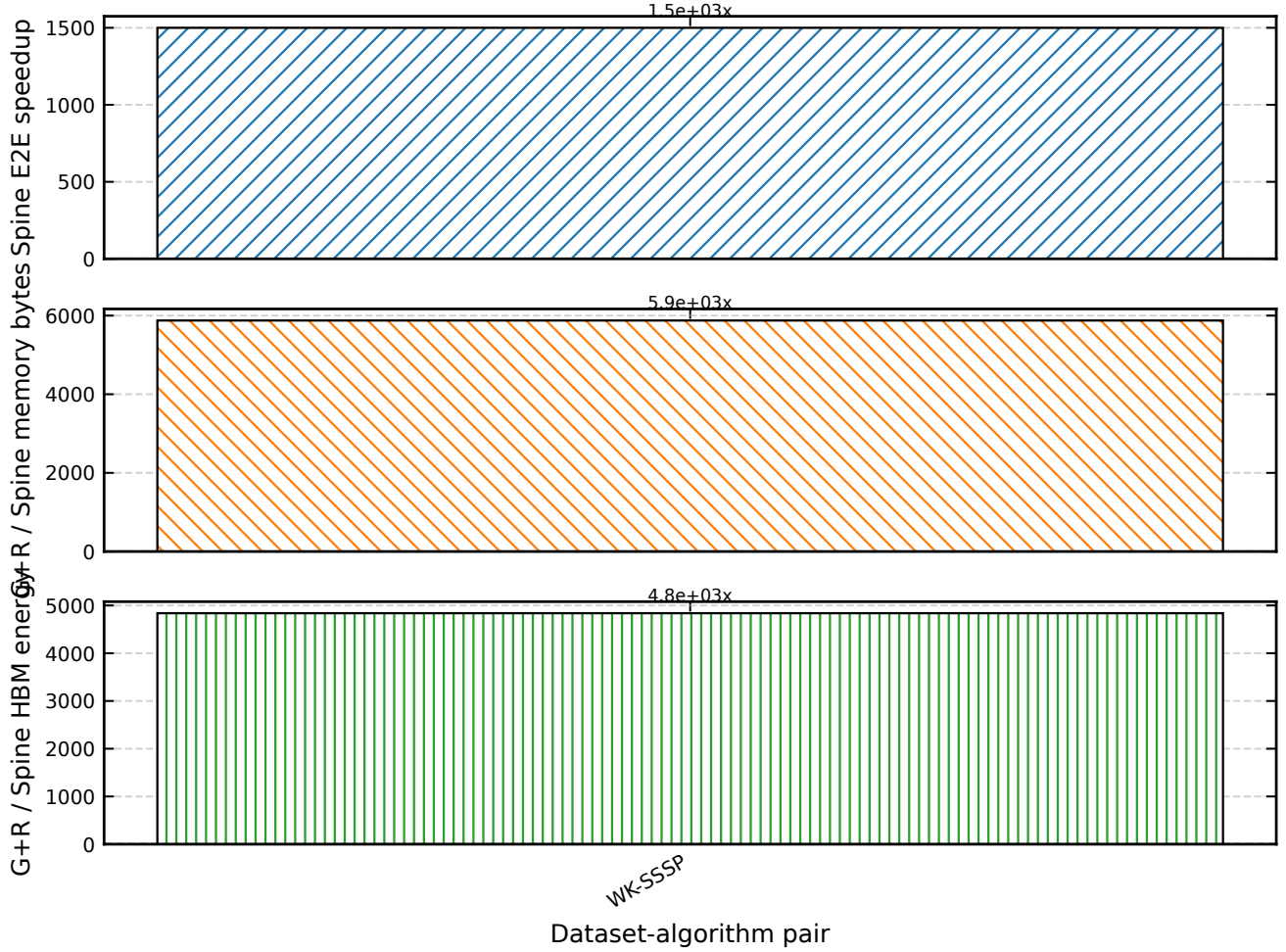


Figure 1: Ratios for complete insertion-batch-8 pairs. Values above one favor Spine for E2E latency and indicate that GraSU+ReGraph uses more memory traffic or HBM energy in the other panels.

Data	Algorithm	Spine cyc	G+R K4 cyc	Speedup
WK	SSSP	43,227	64,837,217	1499.92×

Table 2: Absolute device cycles. Only cross-system final-state-matched pairs enter this table.